

What is network DoS?

- ◆ Goal: take out a large site with little computing work
- ◆ How: **Amplification**
 - Small number of packets $\Rightarrow$ big effect
- ◆ Two types of amplification attacks:
 - DoS bug:
 - ◆ Design flaw allowing one machine to disrupt a service
 - DoS flood:
 - ◆ Command bot-net to generate flood of requests

DoS can happen at any layer

◆ This lecture:

■ Sample Dos at different layers (by order):

- ◆ Link
- ◆ TCP/UDP
- ◆ Application
- ◆ Payment

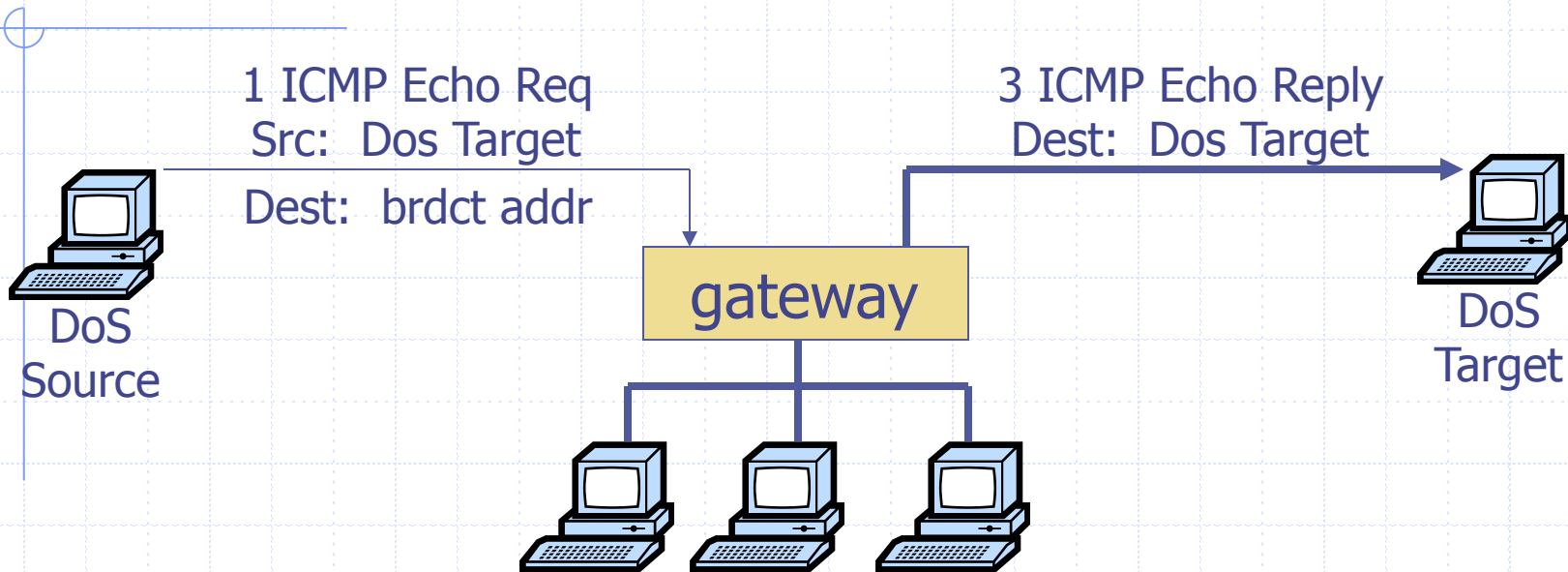
■ Generic DoS solutions

■ Network DoS solutions

◆ Sad truth:

- Current Internet not designed to handle DDoS attacks

Smurf amplification DoS attack

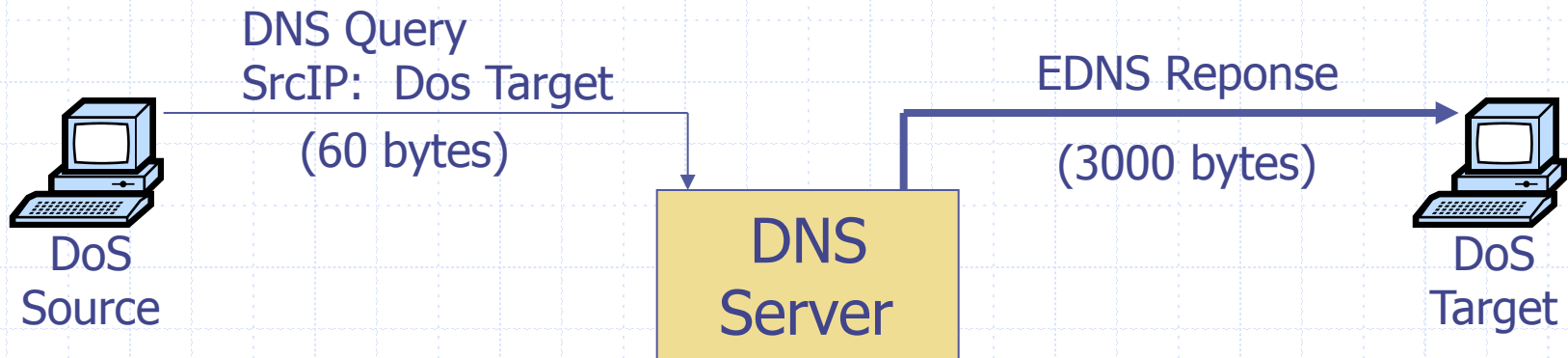


- ◆ Send ping request to broadcast addr (ICMP Echo Req)
- ◆ Lots of responses:
 - Every host on target network generates a ping reply (ICMP Echo Reply) to victim

Prevention: reject external packets to broadcast address

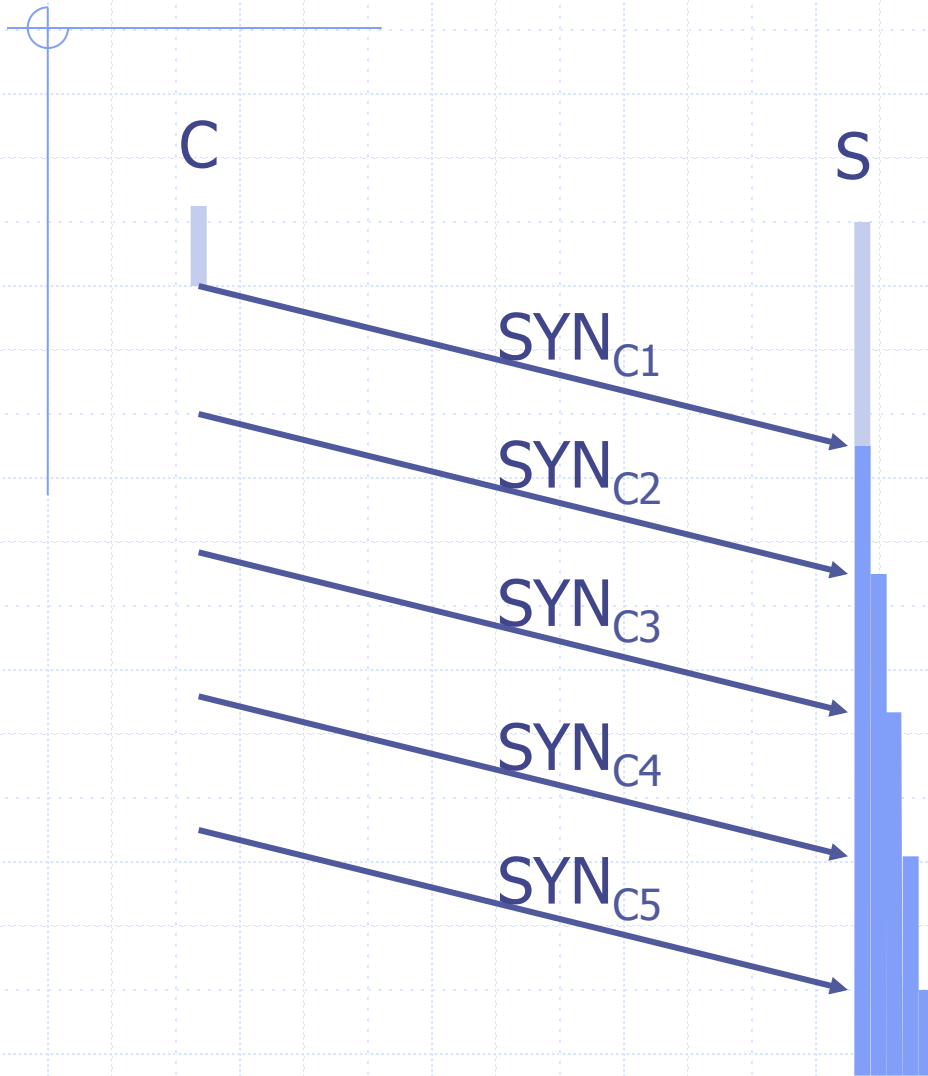
Modern day example (May '06)

DNS Amplification attack: ($\times 50$ amplification)



580,000 open resolvers on Internet (Kaminsky-Shiffman' 06)

TCP SYN Flood I: low rate (DoS bug)



Single machine:

- SYN Packets with **random source IP addresses**
- Fills up backlog queue on server
- No further connections possible

SYN Floods

(phrack 48, no 13, 1996)

OS	Backlog queue size
Linux 1.2.x	10
FreeBSD 2.1.5	128
WinNT 4.0	6

Backlog timeout: 3 minutes

- ⇒ Attacker need only send 128 SYN packets every 3 minutes.
- ⇒ Low rate SYN flood

A classic SYN flood example

◆ MS Blaster worm (2003)

- Infected machines at noon on Aug 16th:
 - ◆ SYN flood on port 80 to **windowsupdate.com**
 - ◆ 50 SYN packets every second.
 - each packet is 40 bytes.
 - ◆ Spoofed source IP: a.b.X.Y where X,Y random.

◆ MS solution:

- new name: **windowsupdate.microsoft.com**
- Win update file delivered by Akamai

Low rate SYN flood defenses

◆ Non-solution:

- Increase backlog queue size or decrease timeout

◆ Correct solution (when under attack) :

- **Syncookies**: remove state from server
- Small performance overhead

SYN Floods II: Massive flood

(e.g BetCris.com '03)

- ◆ Command bot army to flood specific target: (DDoS)
 - **20,000** bots can generate **2Gb/sec** of SYNs (2003)
 - At web site:
 - ◆ Saturates network uplink or network router
 - ◆ Random source IP ⇒
attack SYNs look the same as real SYNs
 - What to do ???

Stronger attacks: TCP con flood

◆ Command bot army to:

- Complete TCP connection to web site
- Send short HTTP HEAD request
- Repeat

◆ Will bypass SYN flood protection proxy

◆ ... but:

- Attacker can no longer use random source IPs.
 - ◆ Reveals location of bot zombies
- Proxy can now block or rate-limit bots.

DNS DoS Attacks (e.g. bluesecurity '06)

- ◆ DNS runs on UDP port 53
 - DNS entry for victim.com hosted at victim_isp.com
- ◆ DDoS attack:
 - flood victim_isp.com with requests for victim.com
 - **Random source IP address** in UDP packets
- ◆ Takes out entire DNS server: (collateral damage)
 - bluesecurity DNS hosted at Tucows DNS server
 - DNS DDoS took out Tucows hosting many many sites
- ◆ What to do ???

DoS at higher layers

◆ SSL/TLS handshake [SD' 03]



- RSA-encrypt speed $\approx 10\times$ RSA-decrypt speed
⇒ Single machine can bring down ten web servers

◆ Similar problem with application DoS:

- Send HTTP request for some large PDF file
⇒ Easy work for client, hard work for server.

1. Client puzzles

◆ Idea: slow down attacker

◆ Moderately hard problem:

- Given challenge C find X such that

$$\text{LSB}_n(\text{SHA-1}(C \parallel X)) = 0^n$$

- Assumption: takes expected 2^n time to solve
- For $n=16$ takes about .3sec on 1GHz machine
- Main point: checking puzzle solution is easy.

◆ During DoS attack:

- Everyone must submit puzzle solution with requests
- When no attack: do not require puzzle solution

2. CAPTCHAs

- ◆ Idea: verify that connection is from a human



- ◆ Applies to application layer DDoS [Killbots '05]
 - During attack: generate CAPTCHAs and process request only if valid solution
 - Present one CAPTCHA per source IP address.

3. Source identification

Goal: identify packet source

Ultimate goal: block attack at the source

Problem: Reflector attacks [Paxson '01]

◆ Reflector:

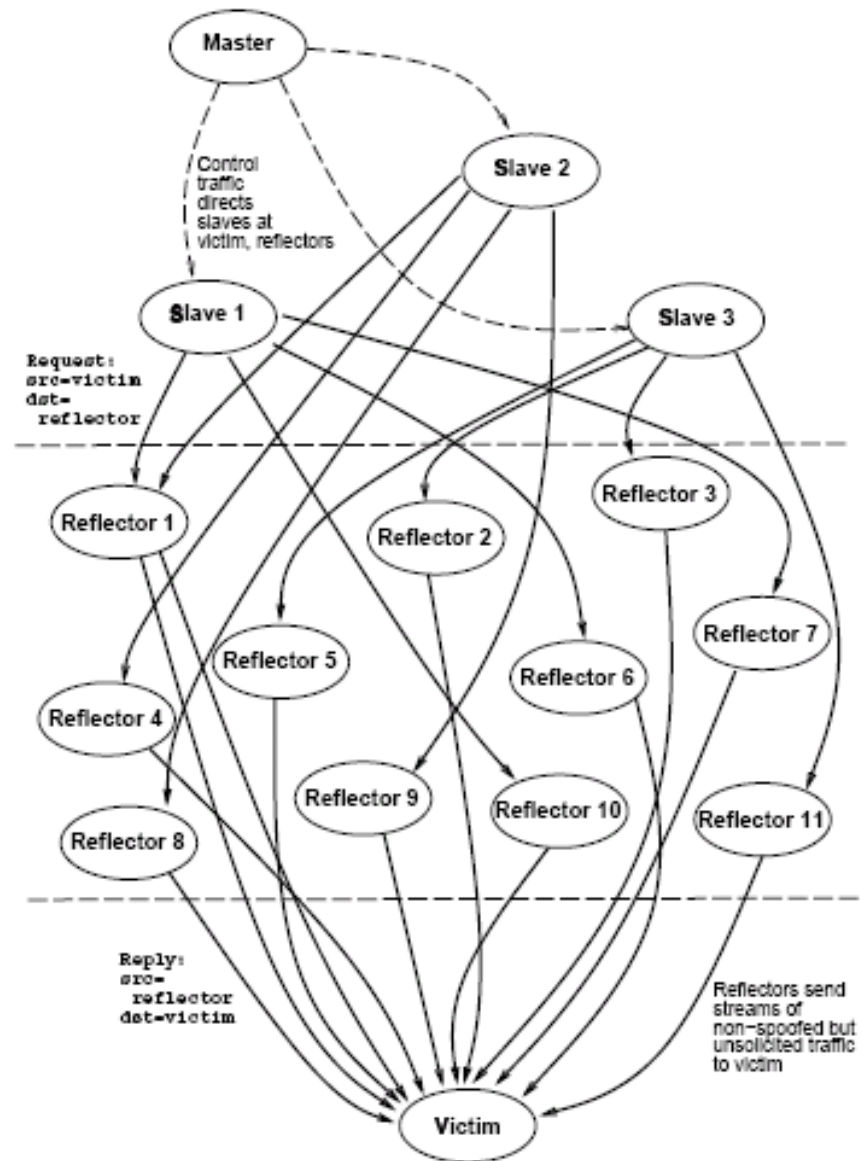
- A network component that responds to packets
- Response sent to victim (spoofed source IP)

◆ Examples:

- DNS Resolvers: UDP 53 with victim.com source
 - ◆ At victim: DNS response
- Web servers: TCP SYN 80 with victim.com source
 - ◆ At victim: TCP SYN ACK packet
- Gnutella servers

DoS Attack

- ◆ Single Master
- ◆ Many bots to generate flood
- ◆ Zillions of reflectors to hide bots
 - Kills traceback and pushback methods



Spam Email

How email works: **SMTP**

(RFC 821, 1982)

◆ Some SMTP Commands:

MAIL FROM: <reverse-path>

Repeated
for each
recipient {

RCPT TO: <forward-path>

RCPT TO: <forward-path>

If unknown recipient: response “550 Failure reply”

DATA

email headers and contents

■

◆ **VERFY** username (Often disabled)

- 250 (user exists) or 550 (no such user)

Spam Blacklists

- ◆ RBL: Realtime Blackhole Lists
 - Includes servers or ISPs that generate lots of spam
 - **spamhaus.org** , **spamcop.net**
- ◆ Effectiveness (stats from spamhaus.org):
 - RBL can stop about 15-25% of incoming spam at SMTP connection time,
 - Over 90% of spam with message body URI checks
- ◆ Spammer goal:
 - Evade blacklists by hiding its source IP address.

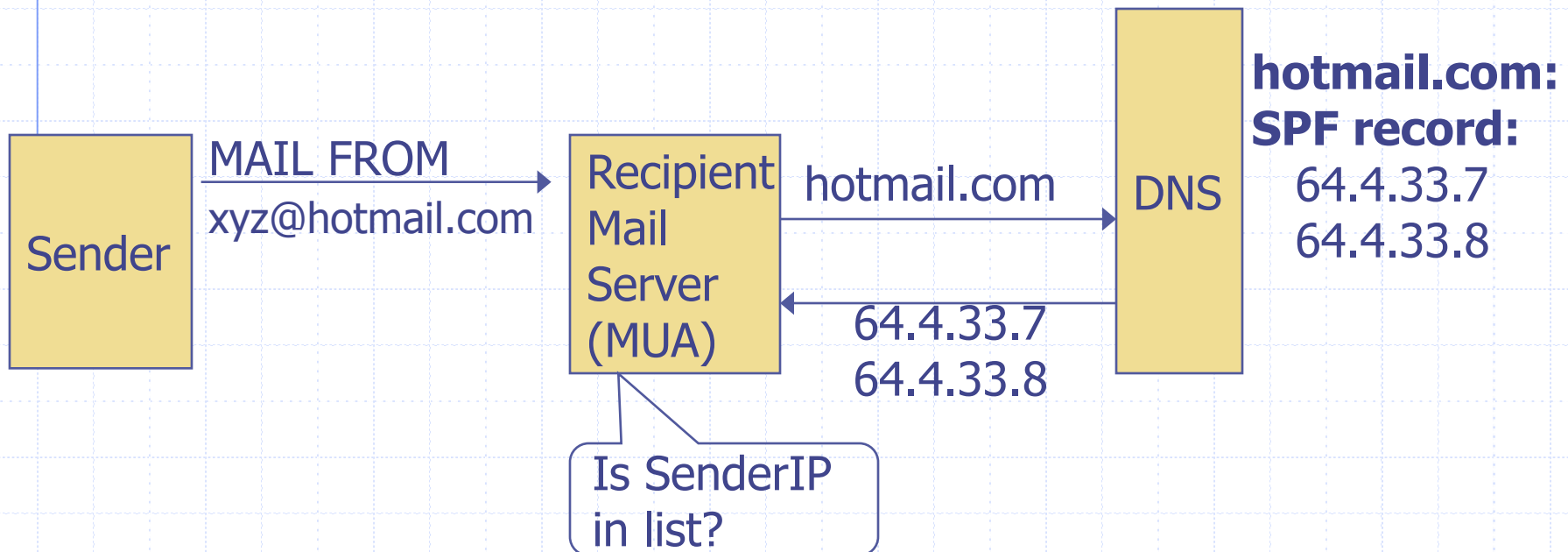
Open relays

- ◆ SMTP Relay forwards mail to destination
 1. Bulk email tool connects via SMTP (port 25)
 2. Sends list of recipients (via RCPT TO command)
 3. Sends email body --- once for all recipients
 4. Relay delivers message

- ◆ Honest relay:
 - Adds **Received** header revealing source IP
 - Hacked relay does not

Sender verification I: SPF

- ◆ Goal: prevent spoof email claiming to be from HotMail
 - Why? Bounce messages flood HotMail system



More precisely: **hotmail.com TXT v=spf1 a:mailers.hotmail.com -all**

Sender verification II: DKIM

- ◆ Domain Keys Identified Mail (DKIM)
 - Same goal as SPF. Harder to spoof.
- ◆ Basic idea:
 - Sender's MTA signs email
 - ◆ Including body and selected header fields
 - Receiver's MUA checks sig
 - ◆ Rejects email if invalid
 - Sender's public key managed by DNS
 - ◆ Subdomain: **domainkey.hotmail.com**

DKIM header example

DKIM-Signature: a=rsa-sha1; q=dns;
d=hotmail.com (domain)
s=may2006; c=relaxed/simple; (selector)
t=1117574938; x=1118006938; (time/exp)
h=from:to:subject:date; (header)
b=dzdVyOfAKCdLXdJOc9G2q8LoXSlEniSb (sig)
av+yuU4zGeeruD00lszZVoG4ZHRNiYzR

- ◆ Recipient's MUA will query for DNS TXT record of
may2006._domainkey.hotmail.com